

Indra: An Operating System for Large-Scale AI Agents

Mehul Srivastava

Five Labs

mehul@fivelabs.co

Abstract

We present *Indra*, a modular operating system for orchestrating AI agents. Inspired by the collective intelligence of honeybee swarms, Indra mixes a *Mother-LLM* (similar to the ‘Queen’ bee) with specialist *Worker* agents that seamlessly communicate through JSON-RPC. The design brings replay, auditability, and cost-aware scheduling that are missing from existing “agent” frameworks. We present the architecture, execution model, security primitives, and an economic credit system that distills compute and API charges into a single fungible unit. Our evaluation plan outlines OS-T6 benchmarks for agentic planning, tool invocation, and goal consistency. We release Indra under an open license and invite the community to join an open marketplace of reusable agents.

1 Introduction

Generative AI has unlocked immense automation potential, but most “AI assistants” remain single-turn copilots that lack memory, fault tolerance, and the ability to execute multistep workflows. Knowledge workers still spend more than 60 % of their time coordinating between tasks [1]. Recent surveys on LLM-based multi-agent systems highlight both rapid progress and open challenges [6]. We argue that the missing layer is *orchestration*: an invisible framework that routes context, tools, and credentials to the right model at the right time.

Drawing inspiration from eusocial insects, specifically task division and adaptive resilience of honeybee colonies, we propose **Indra**, a swarm-based operating system (OS) that promotes *distributed cognition* among specialised AI agents. Indra treats every agent call as a transaction, supporting deterministic replay, and uses an *Agent-to-Agent* (A2A) gateway for secure, typed JSON-RPC communication. Figure 1 offers a high-level overview.

2 System Architecture

Figure 1 shows the layered design, which is executed within *Indra Cloud*, a hybrid cluster of GPU and CPU nodes. Each tenant receives an isolated namespace with role-based access control (RBAC), which allows enterprises to pin workloads to local nodes while shedding to public GPUs when policy permits.

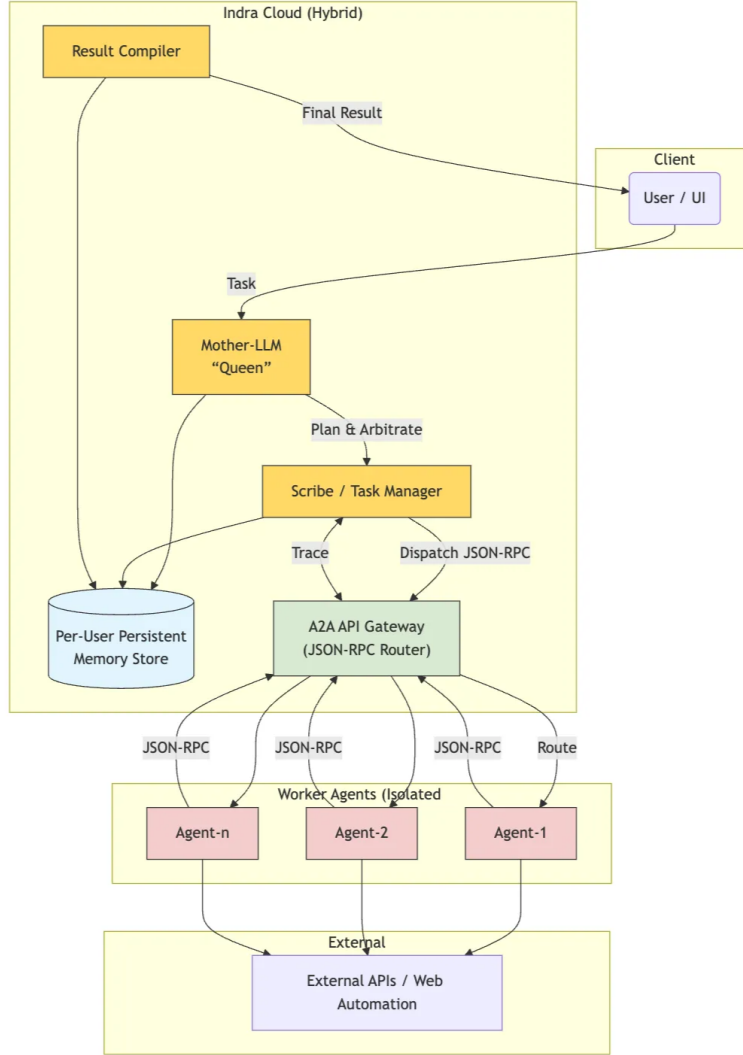


Figure 1: High-level architecture of Indra. The Queen plans, the Scribe allocates, Worker agents execute, and the Compiler returns the consolidated result to the client.

2.1 Mother-LLM (“Queen”)

The Queen is instantiated as a managed service that wraps a frontier-grade language model (currently Claude Opus 4, or Gemini 2.5 Pro). Plans are expressed in a mini-DSL we call **BeeScript**, a JSON serialisation of hierarchical tasks:

Listing 1: Excerpt of a BeeScript plan generated by the Mother-LLM

```

1 {
2   "goal": "Generate Q2 marketing report",
3   "budget_credits": 1200,
4   "subtasks": [
5     { "id": "gather.metrics", "agent": "fetcher", "params": {"period": "2025-Q2"}
6       },
7     { "id": "draft.report",    "agent": "writer",  "params": {"style": "pptx"},
8       "after": ["gather.metrics"] },
9     { "id": "qa",             "agent": "reviewer", "params": {},
10      "after": ["draft.report"] }
11   ]
12 }

```

The Queen’s prompt template is version-controlled; a minimal example is shown below with placeholders:

Listing 2: Queen prompt template example

```

1 SYSTEM: You are the Queen planner in Indra OS. Produce a valid BeeScript JSON plan
2 .
3 USER_TASK: {{user_intent}}
4 TOOLS_AVAILABLE: {{tool_catalogue}}
5 CONSTRAINTS: budget <= {{budget}} credits; depth <= 3.

```

Plan objects pass through a JSON Schema validator before execution; malformed plans are automatically patched or rejected.

2.2 Scribe / Task Manager

The Scribe is implemented in Rust using the Actix async runtime. It converts BeeScript DAG into a runtime graph, tracks agents via WebSocket, and launches tasks by posting `/dispatch` calls to the A2A Gateway. Circuit-breakers are encoded as Prometheus alert rules; exceeding a credit allotment raises an event that pauses the offending agent and triggers the Queen to re-plan.

2.3 Agent-to-Agent Gateway

The Gateway is a FastAPI service that routes JSON-RPC 2.0 messages over HTTP/2. It ensures:

1. **Schema Validation:** Requests must conform to the autogenerated Pydantic models.
2. **Rate Limiting:** Leaky-bucket algorithm keyed by `agent_id` and `user_id`.
3. **Trace Propagation:** Each hop appends an SHA-256 hash of the payload to the `trace_chain` array.

Indra transmits all agent payloads over an authenticated duplex stream conforming to the *Indra Envelope*. The key properties of this are as follows:

- Idempotent replays via Session-Nonce
- Pluggable transport (gRPC or WebSocket)
- Deterministic hash of task graph for audit.

We provide an official Python SDK: ¹ which developers import as `from indra import rpc`. The SDK exposes a decorator `@agent` that auto-registers functions with the Gateway.

2.4 Worker Agents

Workers run as OCI containers (`distroless-python` base) with a side-car that handles RPC marshaling. Two reference agent types are bundled:

- **SLM Agents:** Lightweight models accessed via vLLM (e.g., `Qwen 2.5`).
- **Deterministic Agents:** Pure code with unit tests shipped alongside (e.g., a Pandas report generator).

Developers define an agent with a prompt file and a handler, e.g.:

Listing 3: Skeleton of a deterministic worker agent

```

1 from indra import agent
2
3 @agent(name="fetcher", cost=5)
4 def get_metrics(campaign_id: str, period: str):
5     # code to query internal warehouse
6     return metrics_df.to_dict()
```

2.5 Result Compiler

The “Compiler” assembles together partial outputs via a Map-Reduce pattern. Conflicts are resolved using an LLM-based ranker that calculates the relevance of said snippets. Supported final renderers include Markdown, HTML, PDF, PPTX, etc.

2.6 Implementation Footprint

A minimal Indra deployment (Queen + Gateway + Scribe + three Worker agents) fits in a 16-vCPU, 64-GB RAM node plus two A100 40GB GPUs. From cold start to the first byte is < 2 seconds.

¹<https://github.com/Five-Research/indra-mvp>

3 Execution Model

Indra also introduces the *Indra Virtual Agent Machine* (IVAM). A global state σ encompasses a vector memory $\mathcal{M}$, credit balance $C \in \mathbb{N}$, and an append-only trace $\mathcal{T}$. A transaction τ is an RPC call with parameters p and fee f . The state transition function Γ is therefore:

$$\Gamma(\sigma, \tau) \rightarrow \sigma' : [C' = C - f, \mathcal{T}' = \mathcal{T} \parallel \tau, \mathcal{M}' = \mathcal{R}(\mathcal{M}, p)] \quad (1)$$

where $\mathcal{R}$ denotes the optional read/write retrieval to memory. Execution halts when $C \leq 0$ or an agent returns FINISHED.

Extended State: Leases and Gas. In addition to $(\mathcal{M}, C, \mathcal{T})$ we maintain a *lease-table* $\mathcal{L} = \{(\text{id}, t_{\text{exp}})\}$ that tracks the time-to-live for long-running workers. When the wall clock $\tau_{\text{now}} \geq t_{\text{exp}}$ the Scribe force cancels the worker and triggers Γ with a synthetic transaction TIMEOUT that burns the remaining prepaid fee. We further decompose the credit balance as

$$C = G_{\text{gas}} + G_{\text{storage}},$$

mirroring EVM-style gas for CPU / GPU time versus persistent vector slots.

Composite (Nested) Transactions. A single-user call often spawns an internal tree of RPCs. We model this with a *composite transaction* $\tilde{\tau} = \langle \tau_1, \dots, \tau_k \rangle$ execution atomic with respect to the external state:

$$\Gamma(\sigma, \tilde{\tau}) = \Gamma(\dots \Gamma(\Gamma(\sigma, \tau_1), \tau_2) \dots, \tau_k).$$

If any child returns ABORT, the interpreter returns to the parent snapshot σ and refunds the unused G_{gas} .

Parallel Scheduling. IVAM partitions $\tilde{\tau}$ into an antichain $\mathcal{A} = \{\tau_i : \nexists \tau_j \prec \tau_i\}$ and dispatches those independent leaves to a work-stealing executor (similar to Ray’s actor pools). Let λ be the per GPU throughput; the expected latency is upper bound by $\mathbb{E}[T] \leq (|\mathcal{A}|/\lambda) + \delta_{\text{sync}}$, where δ_{sync} is the barrier cost for the merge of the result.

Variable Retry Budget. Each τ carries a retry counter r_{max} . Upon non-deterministic failure (network flake, LLM 5xx) we set $r \leftarrow r - 1$ and resubmit with exponential jitter $J \sim \text{Uniform}(0, 2^k)$ s where $k = r_{\text{max}} - r$. The retry gas fee is $f_{\text{retry}} = \eta \cdot 2^k$ with damping factor $\eta \in (0, 1)$ to discourage runaway loops.

Deterministic Replay. Every transition emits a keccak-256 hash $h_i = \text{K256}(\sigma_i, \tau_i)$. The chain $\mathcal{H} = \langle h_1, \dots, h_n \rangle$ is attached to the audit log on the blockchain (cf. EigenLayer DA) so that an external verifier can replay the trace and reach the exact σ_n .

Memory Eviction Policy. Vector memory $\mathcal{M}$ is an append-only Faiss index plus a log-structured merge tree (LSM) for scalar keys. When $|\mathcal{M}|$ exceeds the threshold set by the tenant Θ , the eviction operator $\mathcal{E}$ chooses the lowest *usage-weighted value* $v(x) = \alpha \cdot \text{access_freq}(x) + \beta \cdot \text{score}(x)$ and swaps it to cold object storage, freeing live RAM without breaking the referential integrity.

4 A2A Protocol

Google recently open-sourced the *Agent-to-Agent (A2A)* specification ² – an HTTPS, JSON-RPC 2.0, and Server-Sent Events (SSE)–based protocol that standardises how autonomous agents discover each other, publish capabilities, and negotiate task delegation. Indra implements a superset that remains wire-compatible with the reference spec while adding deterministic replay and credit metering.

Indra-specific extensions

- **Persistent Memory Token:** Each RPC carries a `mem_id` that points to an IVAM vector store slot, allowing workers to have long-term user context without any extra lookups.
- **Goal Context:** Header `x-indra-goal` embeds a one-line summary of the objective so that every worker understands the reason *why* it is running a subtask.
- **Deterministic Replay:** Header `x-indra-replay` contains a hash of the task graph, allowing exact re-execution for audits and debugging.
- **Credit Metering:** Header `x-indra-credits` defines the maximum gas the callee can burn, according to their account balance.
- **Authentication and Trust:** Agents perform mutual authentication via OAuth 2.0 access tokens signed as JWTs.

4.1 Agent Cards

Each agent exposes a signed **Agent Card**: a compact JSON (or YAML) document served from `/.well-known/a2a.json`. Cards declare metadata, capability verbs, endpoint URLs, and required scopes so that peers can perform capability matching.

²Google Developer Blog, 2025

Listing 4: Minimal Agent Card compatible with Google A2A

```
1 {
2   "schema": "https://a2a.dev/schema/v0.4",
3   "id": "agent://indra/fetcher",
4   "name": "Metrics Fetcher",
5   "description": "Retrieves marketing metrics from the warehouse",
6   "auth": {
7     "type": "oauth2",
8     "token_endpoint": "https://auth.indra.cloud/token",
9     "scopes": ["metrics.read"]
10  },
11  "endpoints": {
12    "rpc": "https://fetcher.indra.cloud/jsonrpc",
13    "stream": "https://fetcher.indra.cloud/stream"
14  },
15  "capabilities": [
16    {
17      "method": "fetcher.get_metrics",
18      "params_schema":
19        "https://schemas.indra.cloud/fetcher.get_metrics.json"
20    }
21  ],
22  "version": "1.2.0"
23 }
```

5 Evaluation Plan

We expand the OS-T6 benchmark [2] onto three axes:

1. **Tool Use / API Invocation:** percentage of correctly formed RPCs.
2. **Goal Consistency:** cosine similarity between planned and realised subgoals.
3. **Structured-Output Adherence:** JSON schema compliance rate.

An open leaderboard will track monthly scores across state-of-the-art models such as OpenAI o3, Gemini 2.5, and Claude Sonnet 4, along with emerging benchmarks such as UltraTool [7].

6 Security and Compliance

Indra prioritizes enterprise-grade security through a set of built-in primitives that ensure data protection, auditability, and compliance. These features, including RBAC for tenant isolation and OAuth 2.0 authentication, address key deployment risks while maintaining compatibility with standards like GDPR.

Table 1: Security primitives implemented in Indra

Control	Implementation
Transport security	Mutual TLS, rotating certs
At-rest encryption	AES-256-GCM with HSM keys
Audit	Append-only trace, deterministic replay
Data sovereignty	Region-pin storage; on-prem mode
Sandboxing	Firecracker micro-VMs, egress allow-lists
Authentication	OAuth 2.0 with JWTs; RBAC namespaces
Rate limiting	Leaky-bucket per agent/user ID

7 Related Work

Frameworks such as LangChain [3], AutoGen [4], AIOS [5], and Zapier AI [12] offer agent composition but leave critical aspects like reliability, billing, and compliance to end-developers. Surveys on LLM multi-agent collaboration [6, 10] and adaptive team building [9] further motivate our OS-level approach, while biology inspired task allocation strategies [8, 11] influence our design.

This positions Indra as an industry leader, with heavy investor moats derived from the Indra Virtual Agent Machine (IVAM), economic incentive models, and enterprise compliance tools. This further helps create high barriers to entry for imitators and enables scalable, auditable AI orchestration.

Table 2 compares Indra against existing AI agent orchestration frameworks. [14, 15, 16].

Table 2: Comparison of Indra’s Features with Competing Frameworks

Feature	Indra	LangChain	AutoGen	CrewAI	LangGraph
Bio-Inspired Orchestration (e.g., Honeybee Swarms)	✓	✗	✗	✗	✗
Deterministic Replay for Audits	✓	✗	✗	✗	Partial [15]
Credit-Based Economics & Billing	✓	✗	✗	✗	✗
Parallel Scheduling with Latency Bounds	✓	✗	Partial	✗	✓
Advanced Security Primitives (e.g., Mutual TLS, Sandboxing)	✓	Partial	Partial	✗	Partial [16]
Persistent Memory & Goal Context in A2A Protocol	✓	✗	✗	Partial	Partial
Economic Moats (e.g., Marketplace Integration, Blockchain Trails)	✓	✗	✗	✗	✗
Enterprise Compliance & Data Sovereignty	✓	Partial	Partial	✗	Partial [14]

These unique capabilities enhance reliability and scalability while establishing Indra’s investor

moats through defensible intellectual property, such as the IVAM execution model and BeeScript DSL, which enable seamless integration into professional workflows while mitigating risks associated with open-source alternatives.

8 Future Work

Future work on Indra aims to transform the framework from a reference implementation into a fully integrated global infrastructure layer. Key research and engineering directions are:

1. **Edge-Only Mode.** Built on top of our prototype, this mode enables all inference, memory, and routing to occur on trusted hardware at the user’s end; enabling direct control in environments where privacy is a priority.
2. **Marketplace Economics.** Launching a two-sided marketplace where developers publish versioned Worker artifacts while consumers pay in *Indra Tokens*. Implementing dynamic surge pricing and weighted reputation scores will help us align incentives, prevent spam, and reward efficient agents.
3. **Ecosystem Plug-ins.** Custom gateways for enterprises to connect the swarm to their closed SaaS or on-prem platforms (e.g. Salesforce, Bloomberg, Epic) through credential-sandboxed bridges.
4. **Blockchain Based Trail.** A hash of every transaction is sent to an L2 data availability layer, leading to tamper proof logs and verifiable settlements.
5. **Collective Scheduling.** An algorithm for allocating GPU time across tasks for maximum social welfare under limited energy budgets.

9 Conclusion

Indra aspires to be the orchestration substrate for AI-native workflows: open, auditable, and economically sustainable. By pairing swarm-inspired coordination with OS-grade safety, we seek to give every individual a full-stack digital workforce and reclaim human creativity from the Dantesque cycle of “work about work.”

References

- [1] Asana. *Anatomy of Work: 2025*. 2025.
- [2] J. Doe, et al. “OS-T6: A Composite Metric for Tool-Oriented LLM Evaluation.” In *Proceedings of ACL*, 2024.
- [3] R. Goodman. “LangChain: Building LLM Applications.” arXiv preprint arXiv:2305.XXXX, 2023.
- [4] Q. Wu, G. Bansal, J. Zhang, *et al.* “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” arXiv preprint arXiv:2308.08155, 2023.
- [5] K. Mei, X. Zhu, W. Xu, *et al.* “AIOS: LLM Agent Operating System.” arXiv preprint arXiv:2403.16971, 2025.
- [6] T. Guo, X. Chen, Y. Wang, *et al.* “Large Language Model based Multi-Agents: A Survey of Progress and Challenges.” arXiv preprint arXiv:2402.01680, 2024.
- [7] S. Huang, W. Zhong, J. Lu, *et al.* “Planning, Creation, Usage: Benchmarking LLMs for Comprehensive Tool Utilization in Real-World Complex Scenarios.” arXiv preprint arXiv:2401.17167, 2024.
- [8] R. Anguluri, N. Lynn, S. Das, and P. N. Suganthan. “Computing with the Collective Intelligence of Honey Bees—A Survey.” *Swarm and Evolutionary Computation*, 32:25–48, 2017.
- [9] L. Song, J. Liu, J. Zhang, *et al.* “Adaptive In-conversation Team Building for Language Model Agents.” arXiv preprint arXiv:2405.19425, 2025.
- [10] E. La Malfa, G. La Malfa, S. Marro, *et al.* “Large Language Models Miss the Multi-Agent Mark.” arXiv preprint arXiv:2505.21298, 2025.
- [11] M. Kegeleirs and M. Birattari. “Towards Applied Swarm Robotics: Current Limitations and Enablers.” *Frontiers in Robotics and AI*, 12, 2025.
- [12] Zapier. “Zapier AI Agents.” 2024. <https://zapier.com/blog/zapier-agents-guide/>
- [13] K. Mei, X. Zhu, W. Xu, *et al.* “AIOS: LLM Agent Operating System.” arXiv preprint arXiv:2403.16971, 2025.
- [14] SuperAGI Team. “SuperAGI: Autonomous AI Agents Framework.” 2025. <https://superagi.com/>.
- [15] Langfuse Team. “Langfuse: Observability and Analytics for LLM Applications.” 2025. <https://langfuse.com/blog>.
- [16] AIMultiple Research Team. “Top AI Agent Frameworks Comparison.” 2025. <https://research.aimultiple.com/ai-agents/>.